

实验课三：大模型强化学习 副本



本次实验不允许大家使用 AI 辅助编程（包括但不限于直接聊天、Copilot、Vibe Coding）。

最终提交结果会进行脚本核验，如发现出现作弊现象（包括但不限于【使用非指定的大模型进行数据构造/训练/推理】），以零分处理。

1. 实验概述

本实验围绕大模型的强化学习技术展开，重点实践**直接偏好优化（DPO）**和**基于群体相对策略优化的强化学习（GRPO）**两种主流方法。实验的目标是让同学们深入理解如何通过强化学习提升模型在复杂推理任务上的表现，掌握数据构造、奖励设计、在线采样等关键环节。所有实验须在单张32GB显存GPU上完成。在线平台最多上传5次结果。

本次实验分为四个层次：

- **Task 1: DPO实验** —— 使用构造的偏好数据对Qwen3-0.6B模型进行DPO训练，比较不同数据构造方案的效果。
- **Task 2: RLVR的GRPO实验** —— 从零实现GRPO算法，在线rollout生成回答，设计正确性奖励和格式奖励，完成训练并分析训练动态。
- **Task 3: vLLM加速（选做）** —— 在GRPO训练中引入vLLM推理加速，对比训练效率与性能。
- **Task 4: 挑战最优方案** —— 在前述任务基础上深挖优化策略，提交测试集结果冲击leaderboard。

允许使用的工具

可以使用以下基础组件：

- **PyTorch**
- **Transformers**
 - 允许加载模型 (AutoModelForCausalLM)
 - 允许加载 tokenizer (AutoTokenizer)
- **datasets**
 - 用于数据读取与预处理

- **accelerate / torch.cuda**
 - 用于设备管理
- 常规 Python 工具库

2. 论文阅读与分享

需要分享大模型强化学习相关的经典论文

Group 1: DPO的延申工作

[KTO:Model Alignment as Prospect Theoretic Optimization](#)

[Rlaif-v: Open-source ai feedback leads to super gpt-4v trustworthiness](#)

Group 2: GRPO的延申工作

[S-GRPO:Early Exit via Reinforcement Learning in Reasoning Models](#)

[Gdpo: Group reward-decoupled normalization policy optimization for multi-reward rl optimization](#)

Group 3: RL application

[Rlef: Grounding code llms in execution feedback with reinforcement learning](#)

[Grounded Reinforcement Learning for Visual Reasoning](#)

Group 4: Agentic RL

[Agentic reasoning and tool integration for llms via reinforcement learning](#)

[Agentic reinforced policy optimization](#)

3. 实验数据与任务说明

3.1 Countdown数据集

本次实验使用**Countdown**数据集 **raw_train.parquet**（将在课程实验平台上发放数据文件），该数据集源自经典的数字游戏任务：给定3~5个小于100的正整数，以及一个目标值target，要求使用每个数恰好一次，通过加、减、乘、除四则运算（除法需结果为整数）计算出目标值。

- **训练集**：原始数据包含题目及对应target，无思维链或中间推理过程。**训练集的数据是过量的，你需要划分出一个测试集用于模型性能测试，以及根据训练速率选择用于训练的数据集大小。**
- **Leaderboard测试集**：用于评估训练完成后的模型性能，包含题目、对应target以及一个唯一序号。

3.2 数据格式与模型输出要求

训练集读取方式

代码块

```
1 ds = load_dataset("parquet", data_files="./raw_train.parquet", split="train")
```

每条数据的原始格式示例：

代码块

```
1 {  
2   "numbers": [3, 8, 2, 7],  
3   "target": 24  
4 }
```

我们不限定模型在训练/推理时的prompt，你可以自由设计，也可以采用以下prompt：

以44、19、35计算98为例

代码块

```
1 Using the numbers [44, 19, 35], create an equation that equals 98. You can use  
basic arithmetic operations (+, -, *, /) and each number can only be used  
once. Show your work in <think> </think> tags. And return the final answer in  
<answer> </answer> tags, for example <answer> (1 + 2) / 3 </answer>.
```

模型需要将最终答案放于<answer>和</answer>之间，最终答案是一个可以被计算的表达式，不需要包括等号，加减乘除分别为 '+' '-' '*' '/', 可以包含括号，可以交换数字间的顺序：

代码块

```
1 推理过程 <answer> 答案表达式 </answer>
```

例如：

代码块

```
1 <think> 8-2=6, 6*3=18, 18+7=25, 不等于24。尝试另一种：7-3=4, 4*2=8, 8+8=16不对。正  
确解法：8/2=4, 4*7=28, 28-3=25还是不对。再尝试：(7-3)*(8-2)=4*6=24。使用数字7,3,8,2
```

各一次。 </think>

2 <answer> (7-3)*(8-2) </answer>

答案表达式必须使用给定数字各一次，运算顺序清晰。

注：我们不限制<think>和</think>以及其他标签的出现，但是最终的表达式必须放在<answer>和</answer>之间，其中<answer>和</answer>前后可以存在空格或换行符

3.3 Teacher Model

原始数据不包含思维链及计算方法，需要使用更强的模型构造推理链用于训练。本实验指定**Qwen-3-8B** (<https://huggingface.co/Qwen/Qwen3-8B>) 作为教师模型 (Teacher Model)，可用于：

- 为训练数据标注思维链和正确答案表达式。
- 修正待训练模型生成的错误推理过程。
- 构造偏好对 (chosen/rejected) 用于DPO训练。

3.4 待训练模型

本实验统一使用**Qwen3-0.6B** (<https://huggingface.co/Qwen/Qwen3-0.6B>)作为基座模型进行强化学习训练。

3.5 注意

由于输出长度会直接影响模型的性能，因此我们规定：测试中模型输出的最长token数设置为1024

4. Task 0: PPO损失函数实现

课上完成PPO代码，课上需要运行通过

代码块

```
1 import torch
2 import torch.nn as nn
3
4 def compute_ppo_clip_loss(
5     old_log_probs: torch.Tensor, # 旧策略对数概率 (batch,)
6     new_log_probs: torch.Tensor, # 新策略对数概率 (batch,)
7     advantages: torch.Tensor,   # 优势函数 (batch,)
8     clip_ratio: float = 0.2     # PPO clip 阈值
```

```

9  ) -> torch.Tensor:
10      # ===== 代码开始 =====
11
12      # ===== 代码结束 =====
13      return loss
14
15  def validate_ppo_implementation():
16      torch.manual_seed(42)
17
18      old_log_probs = torch.tensor([-0.5, -1.0, -0.3, -0.8], requires_grad=False)
19      new_log_probs = torch.tensor([-0.4, -1.1, -0.25, -0.7], requires_grad=True)
20      advantages = torch.tensor([1.2, -0.8, 0.9, -1.5])
21      clip_ratio = 0.2
22
23      loss = compute_ppo_clip_loss(old_log_probs, new_log_probs, advantages,
24      clip_ratio)
25      total_loss = loss.mean()
26
27      total_loss.backward()
28      student_grad = new_log_probs.grad.clone()
29
30      true_loss = torch.tensor([-1.3262053, 0.7238699, -0.9461439, 1.6577564])
31      true_grad = torch.tensor([-0.3315513, 0.18096748, -0.23653598, 0.4144391])
32
33      loss_correct = torch.allclose(loss, true_loss, atol=1e-4)
34      grad_correct = torch.allclose(student_grad, true_grad, atol=1e-4)
35
36      print(f"损失计算: {loss_correct}")
37      print(f"梯度计算: {grad_correct}")
38
39      # 运行实验校验
40      if __name__ == "__main__":
41          validate_ppo_implementation()

```

$$L_{PPO} = -\mathbb{E}[\min(\frac{\pi_{\text{new}}}{\pi_{\text{old}}} A, \text{clip}(\frac{\pi_{\text{new}}}{\pi_{\text{old}}}, 1 - \epsilon, 1 + \epsilon) A)]$$

5. Task 1: DPO实验

5.1 实验目标

实现DPO（Direct Preference Optimization）算法，利用构造的偏好数据训练模型，使其能输出正确的格式及答案。

5.2 实验步骤

Step 1: 数据构造

原始Countdown数据不包含推理过程和偏好标签，你需要自行构造用于DPO训练的数据。**鼓励尝试多种方案并比较效果**，可选的构造方法包括但不限于：

- **方案A**：不构造推理过程只使用正确的答案和错误的回答生成偏好对。
- **方案B**：使用Teacher Model (Qwen-3-8B-Instruct) 为每条训练数据生成完整的思维链和正确答案，作为chosen；同时可以要求Teacher Model生成一个常见的错误推理作为rejected。
- **方案C**：让待训练模型 (Qwen3-0.6B) 对每个问题生成多个回答（如temperature采样），再由Teacher Model评估回答的正确性和推理质量，将好的回答作为chosen，差的作为rejected。
- **方案D**：先由Teacher Model生成正确推理和答案，再通过规则或另一次提示生成包含逻辑错误、格式错误或计算错误的版本作为rejected。

在保证只使用给定的teacher model和待训练的模型，你也可以使用其他方案构造数据。

构造完成后，数据格式应包含：

代码块

```
1  {
2    "prompt": "题目描述及要求",
3    "chosen": "推理+正确表达式+正确格式",
4    "rejected": "错误推理或错误表达式"
5  }
```

Step 2: 实现DPO训练脚本

不能使用trl等成熟大模型训练框架。需要使用 `torch`、`transformers` 以及必要的库自行实现DPO算法。

核心公式回顾：

$$L_{DPO}(\pi_{\theta}; \pi_{ref}) = -E_{(x, y_w, y_l) \sim D} [\log \sigma(\beta * \log(\pi_{\theta}(y_w|x)/\pi_{ref}(y_w|x)) - \beta * \log(\pi_{\theta}(y_l|x)/\pi_{ref}(y_l|x)))]$$

你需要：

- 加载基座模型 (Qwen3-0.6B) 和参考模型（初始与基座模型相同，在训练中不进行更新）。
- 对每个batch的prompt，分别计算chosen response和rejected response的log概率。
- 实现上述损失函数，并通过反向传播更新模型参数。

Step 3: 训练与验证

- 在构造好的训练集上进行训练。
- 在测试集（从训练集划分出）上评估模型的准确率（表达式是否正确且满足数字使用约束）和格式正确率。
- 记录训练时长、数据量、loss曲线、测试集性能，chosen response，rejected response以及输出平均长度的变化
- 【选做】基于成功/失败按理或遇到的问题进行优化，包括但不限于使用SFT进行两阶段训练，在DPO loss中补充NLL loss来提升模型输出chosen response的概率。

5.3 实验分析

在报告中需要对比不同数据构造方案的效果，包括：

- 构造方案的详细介绍
- Step3中提及的训练过程中的指标变化
- 测试集上的准确率和格式正确率
- 典型成功/失败案例分析

6. Task 2: RLVR的GRPO实验

6.1 实验目标

实现原始GRPO（Group Relative Policy Optimization）算法，用于强化学习训练，使模型在Countdown任务上学会通过自我探索生成正确的推理和答案。你需要实现在线采样、奖励计算、优势估计和策略更新的完整流程。

6.2 实验步骤

Step 1: 实现GRPO基本组件

严禁使用成熟的RLHF框架，需自行实现。核心组件包括：

1. **在线Rollout**：对每个prompt（Countdown题目），从当前策略模型采样生成 G 个回答（group size）。
2. **奖励计算**：设计奖励函数，根据回答分配奖励值。奖励函数至少需要包含：
 - **正确性奖励**：判断 `<answer>` 内的表达式计算结果是否等于target。

- **格式奖励**：检查回答是否严格包含 `<answer>` 标签，内部表达式是否合法、是否使用了所有给定数字各一次。

你可以自行设计更细粒度的奖励函数。

一个经典的奖励函数设计是 $r_{\text{正确性}} + 0.2 \cdot r_{\text{格式}}$ ，格式正确和答案正确的奖励都为1，否则为0

3. **优势计算**：对每组内的 G 个回答，计算每个回答的奖励相对于组内均值的优势：

代码块

```
1 A_i = (r_i - mean(r_group)) / (std(r_group) + epsilon)
```

其中epsilon为1e-8，防止除0操作

4. KL loss的实现为选做。
5. **策略更新**：计算GRPO loss，更新模型参数。需要包含clip操作

Group Relative Policy Optimization In order to save the training costs of RL, we adopt Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which foregoes the critic model that is typically the same size as the policy model, and estimates the baseline from group scores instead. Specifically, for each question q , GRPO samples a group of outputs $\{o_1, o_2, \dots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_{θ} by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)] \frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \epsilon, 1 + \epsilon \right) A_i \right) - \beta \mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) \right), \quad (1)$$

$$\mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) = \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - \log \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - 1, \quad (2)$$

where ϵ and β are hyper-parameters, and A_i is the advantage, computed using a group of rewards $\{r_1, r_2, \dots, r_G\}$ corresponding to the outputs within each group:

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}. \quad (3)$$

Step 2: 实现训练循环

根据实现的基本组件，拼装训练循环，在每一轮迭代中：

- 随机采样一批prompt。
- 对每个prompt，模型生成 G 个回答（可并行采样）。
- 计算每个回答的奖励。
- 计算组内优势。
- 计算GRPO损失，更新策略模型。
- 记录奖励均值、输出序列的熵（entropy）、格式正确率、正确性准确率。

Step 3: 超参数设置与记录

你需要尝试不同的超参数组合（如group size、学习率、rollout时的temperature等），并在报告中汇报：

- 最终使用的超参数。
- 训练过程中奖励曲线（总奖励、正确性奖励、格式奖励）。
- 输出熵的变化（观察模型是否变得过于确定或保持多样性）。
- 每个阶段的时间占比：rollout生成时间、奖励计算时间、模型更新时间的统计。

7. Task 3: vLLM加速（选做）

7.1 实验目标

在GRPO训练中，时间瓶颈通常是模型生成回答（rollout）阶段。本任务要求使用vLLM框架加速推理，并同步更新vLLM引擎中的模型权重，实现高效训练。

7.2 实验步骤

1. **部署vLLM推理引擎**：将当前的策略模型加载到vLLM中，提供批量生成接口。
2. **训练与推理解耦**：在rollout阶段，通过vLLM快速生成 **G** 个回答；奖励计算和优势计算完成后，使用PyTorch更新模型权重。
3. **同步权重**：更新后将新权重同步到vLLM引擎（可定期同步或每步同步）。
4. **对比实验**：在相同的超参数设置下，比较使用vLLM加速前后的：
 - 每轮迭代的耗时（rollout时间、奖励计算时间、更新时间）。
 - 显存占用
 - 奖励变化曲线和输出熵变化。
 - 测试集最终性能。

8. Task 4: 挑战最优方案

在完成前面Task的基础上，你可以自由探索更优的训练方案，冲击leaderboard。允许的探索方向包括但不限于：

- 更精细的奖励函数设计（如逐步奖励、中间步骤验证）。

- 更好的优化算法。
- 更好的数据构造策略。
- 多阶段训练，比如对基座模型进行轻量微调后再做强化学习。

提交要求：在课程实验平台上提交测试集推理结果，每人最多提交5次，取最优成绩。测试集只提供题目，不公布答案。

9. 需要上传的内容

9.1 最终版本的实验报告、脚本、结果合集

截止时间：第七周周日23:59前（5月3日23:59前）

上传内容（压缩包，不超过200MB）：

- **PDF实验报告（不超过四页）（组长姓名_学号.pdf）**
 - 内容结构自由组织，包含但不限于实验方案设计、结果对比、分析结论与关键思考；
- **主线任务过程脚本（/task_scripts）**
- **上传内容说明（readme.md）**
 - 即上传的脚本结构、各个脚本功能、任务实行情况说明。

注意：上传文件的大小不得超过200MB，**不需要上传模型的权重文件和数据，只上传可复现的过程脚本**

9.2 Task 4的Leaderboard推理结果

截止时间：第七周上课前（4月28日18:30前）

提交文件： `task_4_test.jsonl`，每行一个JSON对象，格式为：

代码块

```
1  {"id": 1, "prediction": "xxx<answer> (7-3)*(8-2) </answer>"}
```

注意：预测值需给出**完整的模型输出**，不要只截取 `<answer>` 内的表达式。

10. 补充说明

- 实验平台最多接受5次上传（报告及脚本集）和5次leaderboard提交。
- 请严格遵守模型使用限制，所有生成和训练必须基于指定模型。
- 任何作弊行为（如直接调用未授权的API、使用非指定模型生成答案）将导致零分处理。

祝实验顺利！